

The Raster of Bézier Intersection Neighbourhoods (ROBIN) Method

James T. Griffin

July 24, 2026

Abstract

We describe a simple and efficient method for rendering closed curves of quadratic Bézier segments particularly suited to rendering 3D text in a GPU pixel shader. The computation of winding numbers is split into two parts, an initial step on loading the vector data and whose result is stored in the eponymous raster and a final step that is completed per pixel at draw time. The main feature of this method is that the computation complexity of the draw step is constant with respect to the number of curve segments.

1 Introduction

The simplest method for text rendering simply counts the number of curve segments that cross above a given point, each crossing has a ± 1 parity depending on whether it crosses left to right or right to left and the sum of these parities is the *winding number*. If the winding number is non-zero, that point is in the interior of the curve.

1.1 SLUG

For most applications, the sampled points are in columns, and each column can be computed at once. If the text instead appears in a three dimensional scene, then it is simplest to perform the entire calculation independently for every pixel. This is the approach used by the SLUG algorithm. As an optimisation, the curve segments are indexed in vertical bands, so the algorithm only checks segments that could cross above the sampled point. Hence the efficiency at draw time is limited by the maximal number of vertical crossings in the closed curve.

2 ROBIN

Instead of indexing curves in vertical bands, we index them from square neighbourhoods. At draw time only the curve segments that pass through the square neighbourhood around a sample point are intersected. Choosing a fine enough raster guarantees that only a limited number of curves are intersected at draw time, this number depends on the local geometry of the curve rather than the whole curve.

We now explain how this is possible, i.e. how one can cache the intersections of the curves not included in the square neighbourhood. This requires a change in the winding number computation, instead of crossings above a point, we look at crossings through an L-shaped line. The long part of the L stretches vertically to infinity and we are free to choose the x -coordinate of this vertical part, the short part is the unique horizontal segment that connects the sampled point to the vertical part.

The computed winding number is not changed, but now we can choose the x -coordinate of the infinite vertical segments to align with the raster grid. Results can be cached because if a Bézier segment does not intersect a square neighbourhood and if the vertical part of the L runs along the left edge of that square, then the segment cannot cross either the horizontal short part of the L or the vertical part of the L that intersect the edge of the square. In this case the intersection count is constant across the entire square neighbourhood and can be cached. It can be calculated by choosing any point along the left edge of the square, then the L has a trivial short part and it is the usual count of crossings above that point.

The number that is cached is the sum of all crossings for the Bézier segments that do not intersect the square neighbourhood. The only other information that is cached in the raster is

the list of curve segments that intersect the square. There are many ways to do this and some optimisations, see the implementation for one approach.

We next rigorously define the L-crossing numbers for general Bézier segments that may intersect the square, it is this that is computed in a pixel shader.

2.1 L-crossings

The intuition behind the definition below is that we count the number of crossings over a curve segment while travelling along the L towards the point p , but this is equal to the number of crossings while travelling along the longer path: down the vertical part of the L , then along the infinite horizontal ray leftwards, then back along the horizontal ray all the way to the point p . Although this is a longer path, any crossings on the extra horizontal segment are counted twice with opposite parities and so cancel out.

The L-crossing number $c_{L,x'}(\gamma)(p)$ is defined for a real number x' , a curve segment $\gamma : [0, 1] \rightarrow \mathbb{R}^2$ and point p , as a sum of:

1. the horizontal crossing number $c_h(\gamma)(p)$, i.e. the number of up/down crossings across the horizontal infinite ray to the left of the point p , and
2. the $\lrcorner$ -crossing number $c_{\lrcorner,x'}(\gamma)(p)$ across a doubly infinite path consisting of the upward vertical and leftward horizontal rays out of the point (x', p_y) .

This $\lrcorner$ shape bounds an infinite quadrant $Q_{x',p_y} = \{(x, y) \text{ s.t. } x < x' \text{ and } y < p_y\}$, so the crossing number is defined for any continuous curve $\gamma : [0, 1] \rightarrow \mathbb{R}^2$, by the difference

$$\begin{aligned} c_{\lrcorner,x'}(\gamma)(p) &= \mathbb{1}(\gamma(0) \in Q_{x',p_y}) - \mathbb{1}(\gamma(1) \in Q_{x',p_y}). \\ &= \mathbb{1}(\gamma(0)_x < x')\mathbb{1}(\gamma(0)_y < p_y) - \mathbb{1}(\gamma(1)_x < x')\mathbb{1}(\gamma(1)_y < p_y) \end{aligned}$$

From this definition it is clear that for a closed curve (i.e. $\gamma(0) = \gamma(1)$) the L-crossing number is equal to the usual crossing number and hence the winding number. For segments it is calculated by the usual methods for the horizontal crossing number and step functions depending only on the endpoints.

2.2 Robustness

The dependence on floating point arithmetic is the same as the usual crossing number which means the same robust results as the SLUG algorithm. There is in principle a problem if the endpoint of a segment lies close enough to a vertical reference line, i.e. $p_x \approx x'$ and if the floating point representation seen by the pixel shader differs from that used in the caching in which case the shader may think that $p_x = x'$ and get the incorrect result. To remove this possibility we nudge the segment endpoint control points so that this is guaranteed not to happen, with no visual difference to the result.